

Scalable Knowledge Base Generation for Agentic Tutorials using Sub-Agent Architecture

Bora Ilci
Computer Engineering
Yildiz Technical University
Istanbul, Turkey
bora.ilci@std.yildiz.edu.tr

Kaan Emre Kara
Computer Engineering
Yildiz Technical University
Istanbul, Turkey
kaan.kara@std.yildiz.edu.tr

Abstract—The rapid evolution of Large Language Models (LLMs) has enabled the creation of autonomous coding assistants. However, generating comprehensive documentation and tutorials for large, unseen codebases remains a significant challenge due to context window limitations and the tendency of monolithic agents to hallucinate or lose focus in long-horizon tasks. Standard Retrieval-Augmented Generation (RAG) often fails to capture the holistic structural context required for high-quality pedagogical content.

This paper introduces a novel “Sub-Agents Architecture” (Deep Agents) that addresses these scalability issues by dynamically spawning isolated, task-specific agents. Our approach utilizes a two-phase pipeline: first, a main orchestrator decomposes the codebase and assigns sub-agents to generate a modular Knowledge Base (KB) in isolated workspaces; second, a downstream agent utilizes this pre-computed KB to generate high-fidelity tutorials. By decoupling reasoning from execution and enforcing context isolation, our system significantly reduces context pollution and improves the accuracy of generated content. We compare our approach against standard monolithic ReAct agents on three contamination-free repositories, demonstrating superior performance in fidelity, pedagogical quality, and structural coverage.

Index Terms—Agentic AI, Large Language Models, Sub-Agents, Knowledge Base Generation, Automated Documentation, Context Isolation.

I. INTRODUCTION

The landscape of Artificial Intelligence has shifted dramatically from static Large Language Models (LLMs) to “Agentic AI” systems capable of perceiving, reasoning, and acting within dynamic environments. While early paradigms like Chain-of-Thought (CoT) [1] enhanced reasoning capabilities, they lacked the ability to interact with external tools. The introduction of ReAct (Reasoning and Acting) [2] bridged this gap, allowing models to interleave thought processes with API calls to ground their responses in reality.

Despite these advancements, applying monolithic agent architectures to large-scale software engineering tasks presents insurmountable challenges. A single agent attempting to read and understand thousands of files inevitably hits context window limits and suffers from increasing hallucination rates as the task horizon extends. This limitation reflects the core insight of “The Bitter Lesson” by Rich Sutton [8]—that general-purpose methods that scale with computation (like search and massive learning) eventually outperform robust

human-designed approaches (like careful prompt engineering). In the context of coding agents, relying on a single agent’s context window (manual design constraint) is insufficient; we instead need a scalable architecture that leverages compute—in the form of multiple autonomous agents—to search and process information effectively.

To address these limitations, we propose a scalable **Sub-Agents Architecture**. Inspired by modular frameworks like ReWOO [3] and collective intelligence models like Mixture-of-Agents (MoA) [5], our approach decouples the “understanding” phase from the “generation” phase. A central orchestrator dynamically spawns worker agents, each assigned to a specific directory or component. These sub-agents operate in strict context isolation, writing their findings to a virtual filesystem manifest. This method not only bypasses context constraints but also enhances security through scoped tooling.

The key contributions of this paper are:

- 1) **Formal Problem Definition:** A mathematical formulation of the context bottleneck in automated documentation.
- 2) **Sub-Agents Architecture:** A novel implementation of dynamically spawned, context-isolated agents for infinite-context analysis.
- 3) **Secure Scoped Tooling:** A robust implementation of a virtual filesystem that enforces path security, preventing “jailbreaking” by autonomous agents.
- 4) **Rigorous Evaluation:** An empirical study on three real-world repositories (≥10k LOC) comparing our approach to standard ReAct agents using a bespoke Pedagogic Quality Score (Q).

II. RELATED WORK

A. Reasoning and Acting Paradigms

The integration of reasoning with tool use is foundational to modern agents. **ReAct** [2] demonstrated that interleaving reasoning traces with actions reduces error propagation and hallucination compared to reasoning alone. However, ReAct’s serial nature is token-inefficient and prone to context overflow in complex tasks. **ReWOO** [3] addresses this by decoupling reasoning from observation, allowing a planner to define a blueprint that workers execute efficiently, a modularity we adopt in our architecture.

B. Self-Correction and Reflection

Autonomous agents require robustness against errors. ****Re-flexion**** [4] introduced verbal reinforcement learning, allowing agents to maintain an episodic memory of mistakes to avoid repetition without expensive weight updates. ****CRITIC**** [6] further enhances reliability by enabling agents to verify their own outputs using external tools. Our system implicitly employs these concepts by allowing the Main Agent to review and retry sub-agent tasks upon failure.

C. Multi-Agent Systems

Collaborative intelligence is a key frontier. ****Mixture-of-Agents (MoA)**** [5] showed that a layered architecture of open-source models could outperform a single proprietary SOTA model by synthesizing diverse outputs. Similarly, ****CAMEL**** [7] utilized role-playing agents to explore complex tasks autonomously. Our work extends these concepts by implementing heavy context isolation, where "collaboration" occurs through a shared file-system protocol rather than chat, minimizing context pollution and establishing a persistent "Organizational Memory" in the form of the Knowledge Base.

III. METHODOLOGY

A. Problem Formulation

Let C be a codebase consisting of a set of files $\{f_1, f_2, \dots, f_n\}$. The total token count of the codebase is denoted as $|C|$. Let A be an LLM-based agent with a maximum context window W . The "Context Bottleneck" problem arises when $|C| \gg W$. In this scenario, a monolithic agent cannot ingest the entire codebase to generate a tutorial T .

Our goal is to generate a tutorial T that maximizes the ****Fidelity Function**** $F(T, C)$, subject to the constraint that at no point in time t does the agent's active context c_t exceed W :

$$\forall t, |c_t| < W$$

We solve this by decomposing C into disjoint subsets $C_1, C_2, \dots, C_k$ such that each $|C_i| \ll W$, and assigning a sub-agent A_i to process each subset.

B. Phase 1: Knowledge Base Generation

The primary challenge in analyzing large repositories is "context pollution," where irrelevant information obscures critical logic. We address this with a strictly hierarchical approach.

1) *Orchestration Layer*: A Main Agent (implemented as DeepAgent) acts as the project manager. It utilizes a deterministic "folder-walking" strategy to decompose the problem (Algorithm 1).

2) *Worker Sub-Agents*: Each sub-agent (implemented as ToolCallingAgent) is a transient entity spawned for a single task. Key design principles include:

- **Context Isolation ("One-Way Mirror")**: The Main Agent never sees the sub-agent's raw observations or reasoning traces. It only receives a pointer to the sub-agent's output file. This guarantees that the Main Agent's context cost $Cost(M)$ is independent of $|C|$.

Algorithm 1 Deterministic Decomposition Strategy

```

Root ← Codebase Root
Tasks ← []
for Dir in Root.subdirectories() do
  if Dir is source code (e.g. src/) then
    Tasks.append(SpawnAgent(Dir))
  end if
end for
return Tasks

```

- **Sequential Execution**: Sub-agents run sequentially using a robust loop. While slower than parallel execution, this eliminates race conditions and simplifies debugging.
- **Scoped Virtual Filesystem**: Each sub-agent is assigned a private workspace. It has read-only access to the codebase but can only write to its specific sandbox.

3) *Data Persistence (File-Based Database)*: Unlike traditional systems relying on vector databases (RAG) or SQL, our architecture persists knowledge states directly as structured Markdown files in a `data/deep_agent` directory. This "File-Based Database" approach allows agents to natively read and process their own memory without complex ORM layers, reducing cognitive load and allowing for human inspectability.

C. Phase 2: Tutorial Generation

The second phase utilizes the generated Knowledge Base to produce user-facing tutorials. By separating the "understanding" phase (KB generation) from the "teaching" phase, the Tutorial Agent can retrieve precise, pre-digested context from the KB rather than raw code, significantly reducing hallucination and improving pedagogical flow.

IV. SYSTEM IMPLEMENTATION

We implemented the system using the **Smolagents** framework for agent orchestration and **LiteLLM** for model abstraction. **Phoenix** (OpenTelemetry) was used for tracing and debugging.

A. The `spawn_sub_agents` Tool

The core of our implementation is the `spawn_sub_agents` tool. It dynamically creates isolated environments for workers.

- **Input**: A list of task descriptions and directory targets.
- **Process**: It iterates through tasks, instantiating a new agent for each. It injects scoped file-system tools restricted to the specific target directory.
- **Output**: A JSON manifest mapping each agent ID to its output workspace path.

B. Path Security and Isolation

To prevent "jailbreaking" where an agent might modify the original codebase or access unauthorized files (e.g. `/etc/passwd`), we implemented strict path validation in `toolkits/scoped_filesystem_toolkit.py`. All file operations run through a proxy that:

- 1) Resolves all paths to their absolute form using `os.path.abspath` and `os.path.realpath`.
- 2) Verifies that the resolved path starts with the allowed workspace prefix.
- 3) Rejects any operation containing traversal patterns (e.g., `../`).

This "Sandbox-by-Design" ensures that even if an agent hallucinates malicious commands, the underlying system remains secure.

V. EXPERIMENTAL SETUP

To validate the efficacy of our Sub-Agents Architecture, we conducted a comparative study against a strong baseline.

A. Dataset

We selected three "contamination-free" repositories—codebases created after the knowledge cutoff of our underlying models—to prevent memorization bias. The repositories were:

- 1) **DeepAgents**: A complex multi-agent framework ($\approx 15k$ LOC). High complexity with multiple inter-dependencies.
- 2) **TOON**: A data serialization library ($\approx 10k$ LOC). Medium complexity, algorithmic focus.
- 3) **Agent Lightning**: An agentic workflow tool ($\approx 12k$ LOC). Medium complexity, application focus.

B. Baselines

We compared two configurations:

- **Baseline Agent**: A standard ReAct agent equipped with RAG (vector search) and file-search tools, but operating as a single monolithic entity.
- **DeepAgent (Ours)**: The proposed two-phase architecture with 4-6 sub-agents per repository.

C. Evaluation Metrics

We employed a multi-faceted evaluation strategy:

1) *Pedagogic Quality Score (Q)*: We defined a composite metric for human evaluation:

$$Q = 0.5a + 0.3b + 0.2c \quad (1)$$

Where:

- *a* (1-5): **Explanatory Completeness**. Does it explain "Why" and not just "What"?
- *b* (1-5): **Code Example Relevance**. Are the examples runnable and idiomatic?
- *c* (1-5): **Visual Support**. Does it correctly use Mermaid diagrams to illustrate architecture?

2) *Structural Coverage*: The ratio of correctly identified and documented components to the total components in the repository.

3) *Context Efficiency*: We measured the **Context Overflow Rate** (number of times the agent hit the token limit) and the ****Total Tokens per Line of Code (Tokens/LoC)****.

D. Ablation Study: The Impact of Hierarchy

To validate our claim that the hierarchical sub-agent search is critical, we compared our full method against an ablated version ("Direct Generation") which omits the Knowledge Base phase. In this version, the Tutorial Agent attempts to generate tutorials directly from the codebase using RAG, effectively simulating a standard "Chat with Codebase" workflow.

TABLE I: Ablation Study Results (Agent Lightning Repo)

Method	Win Rate	Overflows	Avg Cost (Tokens)
Full Architecture	75%	0	150k
No KB (Direct)	40%	3	110k

Table I shows that while the Full Architecture incurs a 36% higher token cost, it is necessary for maintaining context safety and high quality (75% win rate). The "No KB" approach frequently hallucinated APIs that didn't exist or missed critical setup steps, proving the value of the "Intermediate Knowledge State" (the KB) as a grounding mechanism.

VI. RESULTS

A. Quantitative Performance

Table II summarizes the win-rates from our Expert Evaluation (N=3 Senior Engineers). The DeepAgent architecture significantly outperformed the Baseline across all metrics.

TABLE II: Expert Evaluation Win-Rates (N=3 Judges)

Metric	Baseline	DeepAgent	Tie
Fidelity	15%	75%	10%
Pedagogy (<i>Q</i>)	20%	70%	10%
Coverage	10%	85%	5%

The most striking result is in **Coverage**. The Baseline agent frequently "gave up" after analyzing deeply nested directories, whereas the Sub-Agents systematically covered every target directory, ensuring no component was left undocumented.

B. Efficiency and Cost

Table III presents the cost analysis. While DeepAgent consumes more tokens overall (due to the overhead of spawning multiple agents), it completely eliminates context overflows.

TABLE III: Cost and Efficiency Analysis (Average per Repo)

Metric	Baseline	DeepAgent
Total Output Tokens	120k	150k (+25%)
Context Overflows	2.3	0.0
Tool Calls	45	12 (Main) / 140 (Total)

The "Cost of Quality" trade-off is evident: we spend 25% more tokens to achieve 0% failure rate and 85% coverage win-rate. This validates the "Bitter Lesson" hypothesis: spending compute (tokens) is more effective than optimizing the prompt for a single agent.

C. Qualitative Analysis

To understand *why* our approach succeeds, we present a case study from the **Agent Lightning** repository.

Case Study: API Layer Documentation

Challenge: The repository contained a complex set of dynamic API endpoints defined via decorators, not explicit class methods.

Baseline Failure: The Baseline Agent missed the dynamic endpoints entirely, documenting only the base API class. It hallucinated standard REST methods (`get_users`) that did not exist in the code, assuming a generic API structure.

DeepAgent Success: The Sub-Agent assigned to `'src/api'` read the decorator logic file first, understood the pattern, and correctly listed the dynamic endpoints in its summary. The resulting tutorial explained *how* to register new endpoints using this pattern, demonstrating deep semantic understanding rather than surface-level pattern matching.

VII. CONCLUSION

In this paper, we introduced a Sub-Agents Architecture for automating the generation of tutorials for large codebases. By decoupling reasoning from execution and implementing strict context isolation, our system overcomes the "context window bottleneck" that plagues monolithic agents. Our experiments demonstrate that this approach not only produces more accurate and comprehensive documentation but also offers a scalable path forward for autonomous software engineering agents. Future work will explore parallel execution of sub-agents and integration with active-coding environments for real-time documentation updates and test generation.

REFERENCES

- [1] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in NeurIPS, 2022.
- [2] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in ICLR, 2023.
- [3] X. Xu et al., "ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models," arXiv:2305.18323, 2023.
- [4] N. Shinn et al., "Reflexion: Language Agents with Verbal Reinforcement Learning," in NeurIPS, 2023.
- [5] J. Wang et al., "Mixture-of-Agents Enhances Large Language Model Capabilities," arXiv:2406.04692, 2024.
- [6] Z. Gou et al., "CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing," in ICLR, 2024.
- [7] G. Li et al., "CAMEL: Communicative Agents for 'Mind' Exploration of Large Scale Language Model Society," in NeurIPS, 2023.
- [8] R. Sutton, "The Bitter Lesson," Incomplete Ideas (Blog), 2019.

APPENDIX

To facilitate reproducibility, we provide the core system prompts used in our architecture.

A. Sub-Agent Knowledge Base Prompt

This prompt guides the "Researcher" agents to analyze code and produce structured summaries.

```
You are a Senior Technical Documentation Specialist.  
Your goal is to produce a definitive, high-quality analysis  
...  
  
Execution Workflow:  
PHASE 1: DISCOVERY
```

```
1. Skeleton Scan: Use 'read_python_structure'...  
2. Read Content: Use 'read_codebase_file'...  
3. Analyze Context: Understand responsibility...
```

PHASE 2: SYNTHESIS

```
4. Draft Content: Mentally compose content.  
- Rule: Include mermaid diagrams.  
- Rule: NO PLACEHOLDERS. Use REAL names.  
- Rule: VERIFY IMPORTS.
```

PHASE 3: OUTPUT

```
5. Write File: Call 'write_workspace_file' exactly ONCE.
```

B. Tutorial Agent Prompt

This prompt instructs the downstream agent to generate pedagogical content using the Knowledge Base.

```
You are a Senior Developer Advocate.  
Your goal is to teach developers how to build something  
specific...
```

Execution Workflow:

PHASE 1: RESEARCH

```
1. Gather Context: Use 'read_knowledge_base_file'...  
2. Verify Code: Use 'read_codebase_file'...  
- CRITICAL RULE: Verify every path exists.
```

PHASE 2: LESSON PLANNING

```
3. Structure: Design logical flow.  
- Start with "Why".  
- Architecture First (Visuals).
```

PHASE 3: WRITING

```
4. Write File: Call 'write_workspace_file'.  
- Rule: Code must be runnable.  
- Rule: Explain why, not just what.
```